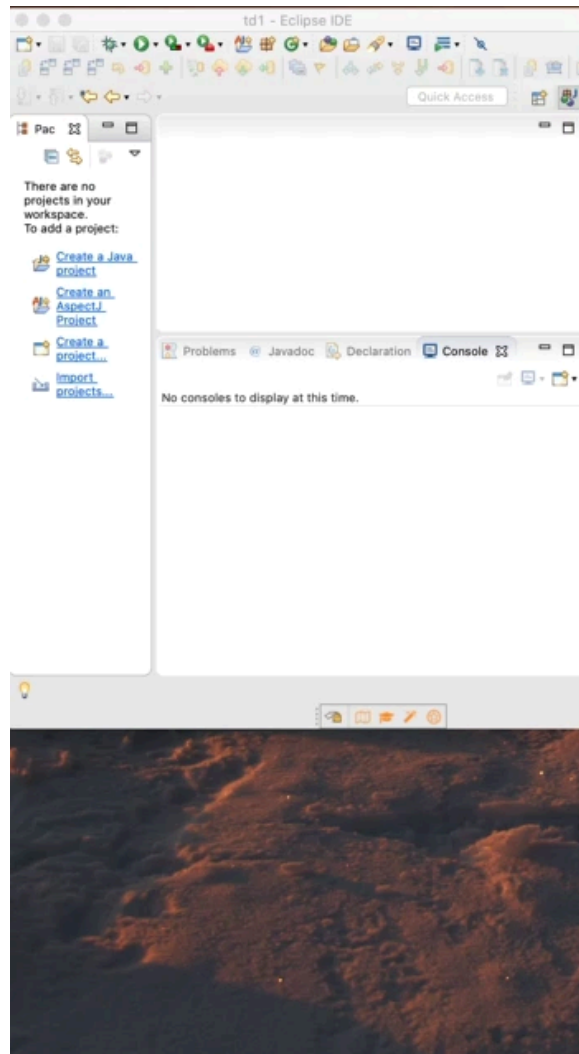


Le but de cet exercice est de faire en sorte qu'une application puisse passer à l'échelle. Or, cette application possède un état ; dans l'exemple qui nous occupe, l'application est une application web en Java / Spring qui a pour état la session indiquant quel est l'utilisateur.

Mise en route

Vous partirez du code disponible sur le dépôt git <https://svn.ensisa.uha.fr/scm/git/todo-public>

La procédure de mise en route en vidéo (vous utiliserez la cible maven jetty:run au lieu de tomcat7:run) :



Importez ce projet sous Eclipse en tant que "General Project". Il s'agit d'un projet Maven, vous aurez donc, une fois importé, à l'indiquer à Eclipse avec le menu contextuel "Configure" / "Convert to Maven project". Il s'agit d'un projet multi-modules : le POM référence les sous-modules (model, dao-mem, dao-sql et app) ; on peut donc installer tous ces modules à l'aide du menu contextuel sur le projet principal "Run as" / "Maven build..." ; indiquez **install** dans "Goals".

Vous aurez à intervenir sur le module "app". Pour plus de confort, matérialisez le projet dans Eclipse : utilisez le menu contextuel sur le répertoire app, "Import..." / "Existing Maven Projects". Vous pourrez à ce moment oublier le premier projet.

Pour tester l'application (dans le répertoire app) `mvn jetty:run` (ou "Run as" sur app / Maven build... / Goal : jetty:run) lance le service sur <http://localhost:8080/>.

Dans ce projet, la gestion des sessions est faite par la classe `fr.uha.ensisa.ff.todo.app.config.SessionInterceptor`. La méthode `preHandle` est invoquée avant tout appel à un contrôleur et peut empêcher l'exécution de ce dernier si la méthode retourne false. Un `cookie` est ici utilisé pour jouer le rôle d'un jeton d'identification. Si `preHandle` ne trouve pas le cookie (de nom `sid`), ou si aucune session ne peut être créée, l'application renvoie une erreur 401.

retrouvée à l'aide du contenu de ce cookie, ou si la session retrouvée est trop ancienne, alors l'utilisateur est redirigé sur la page d'authentification /auth. La création d'un cookie est elle gérée par le contrôleur `fr.uha.ensisa.ff.todo.app.controller.SIDController`, sur invocation (POST) de la page d'authentification.

Les sessions sont, côté serveur, enregistrées dans une `ConcurrentHashMap` (une HashMap réagissant de manière sûre à plusieurs threads concurrentes) initialisée par `fr.uha.ensisa.ff.todo.app.config.MvcConfiguration::getSessions`, et distribuée au `SessionInterceptor` et au `SIDController` à l'aide de l'annotation `@Autowired`.

Votre but est donc de remplacer cette `ConcurrentHashMap` par une autre implémentation de `Map` qui soit distribuée, c'est à dire que plusieurs instances de la même application devront partager.

Partage des sessions à l'aide d'Hazelcast

Vous n'aurez à intervenir que sur le module app.

[Hazelcast](#) est un outil très complet lorsqu'on en vient à gérer divers aspects d'une application distribuée. Il permet par exemple de gérer des locks, des exécutions, ou des maps distribuées. Lorsqu'on travaille avec Java, il n'est d'ailleurs pas même nécessaire d'installer de nouveaux services, car l'outil peut être embarqué dans l'application. Le principe de base est que lorsqu'une instance d'Hazelcast démarre, elle tente de se connecter à des pairs. Elle les trouve par défaut à l'aide d'un message broadcast (y a-t-il d'autres instances d'Hazelcast ici ?), mais d'autres mécanismes (plus sûrs) sont aussi possibles (connection à certains membres connus, recherche dans un annuaire etcd ou autre, intégration à un cloud public...). Une fois ses pairs retrouvés, elle forme avec eux un cluster capable de réaliser ensemble les fonctionnalités évoquées plus haut. Par exemple, la possibilité de stocker sur plusieurs nœuds une map accessible à tous les nœuds. C'est ce qui va nous intéresser ici.

Vous vous réferez à la [documentation d'Hazelcast](#) que vous pourrez intégrer au projet en [l'ajoutant comme dépendance](#) dans le pom.xml du module app. Lancez bien un [membre](#) Hazelcast et pas juste un client.

Modifiez le code pour que les sessions enregistrées dans la Map créée par `fr.uha.ensisa.ff.todo.app.config.MvcConfiguration::getSessions` du module app soit une `Map` Hazelcast. Attention, à bien donner un nom à la map créée, nom qui permettra aux différentes instances de l'application qu'il s'agit là bien d'une seule et même map distribuée (on pourrait en avoir plusieurs).

Pour lancer plusieurs instances de tomcat et ainsi voir que les sessions sont bien partagées, utilisez `mvn tomcat7:run` en modifiant le port de lancement déterminé par la propriété `servlet.port` du pom.xml. On pourra simplement modifier ce port en lançant par exemple la commande `mvn jetty:run -Dservlet.port=8081` (fonctionne aussi avec `Run as / Maven build...` / `Goal jetty:run -Dservlet.port=8081`). Vérifiez bien à chaque fois que les différentes instances constituent bien un cluster Hazelcast (voir les logs). Notez que vous aurez peut-être à ajouter l'option `-Djava.net.preferIPv4Stack=true` pour que les instances d'Hazelcast parviennent à se voir entre elles. Si tout se passe bien, se connecter sur une instance de l'application devrait faire en sorte d'être connecté à l'autre (attention à ne pas tester sur la page /auth) ; de même, se déconnecter sur une instance devrait déconnecter l'utilisateur de l'autre. Attention, la durée de session est configurée à 25s (voir `SessionInterceptor::SessionTimeoutS`). Ne vous attendez pas non plus à ce que les tâches soient partagées entre les instances : c'est le dao-mem qui est utilisé et toutes les tâches résident dans la mémoire du nœud où elles ont été créées (et non dans la map des sessions) ; il faudrait lancer une base de données MySQL ou Postgresql pour que cela fonctionne.

Sauvegarde des données et efficacité

Si les différents membres Hazelcast forment un cluster, la notion de "nuage" ou "cloud" peut venir à l'esprit. Mais quand il en vient à stocker effectivement des données, il faudra utiliser la mémoire physique d'un des membres. Or, si un de ces membres vient à disparaître, il ne faudrait pas perdre ces données. Pour cela, il serait nécessaire de faire une copie de ces données. Configurez maintenant une [réplication](#), de préférence asynchrone (une copie se fera en arrière-plan, sans bloquer le processus faisant une mise à jour). Vous préférerez une [configuration programmatique](#). Donnez alors une configuration (`com.hazelcast.config.Config`) que vous passerez en paramètre à `Hazelcast::newHazelcastInstance`, et dans laquelle vous aurez ajouté une `com.hazelcast.config.MapConfig` pour la map que vous aurez utilisé. Attention, la configuration de la map doit avoir le même nom de la map configurée. Supprimer un membre ne devrait alors pas déconnecter un utilisateur.

Il est également inutile de conserver les données de sessions au-delà de leur durée de vie. Vous pouvez ainsi établir un [TTL](#) d'un peu plus que la durée de la session (attention à bien remettre la session à jour pour renouveler le bail).

Enfin, en admettant qu'on utilise de nombreuses instances de l'application, les données d'une session ne seront localisées que dans 2 nœuds (le nœud principal et sa réplique). En admettant qu'on aie mis en place un répartiteur de charge avec affinité de session (session stickiness), si l'instance effectivement utilisée ne correspond pas au membre Hazelcast qui stocke la session, une requête réseau sera lancée vers le bon membre pour la récupérer, et ce à chaque requête. Pour limiter l'usage du réseau, utilisez un cache qui permettra de stocker en local pour une durée donnée (de l'ordre de la durée de la session) les informations de la session. Hazelcast appelle cela un ["near cache"](#).

Conclusion

Il existe un [moyen plus simple](#) de gérer les sessions à l'aide d'Hazelcast. De plus, il serait plus pertinent d'utiliser une librairie comme [spring security](#) pour gérer une authentification ; de manière générale, il ne faut jamais implémenter une solution de sécurité soi-même, sauf à être un spécialiste et contribuer à une bibliothèque spécialisée.

Cependant, vous aurez expérimenté dans cet exercice une manière relativement simple de partager un état entre des processus, potentiellement localisés sur des machines différentes. Ainsi, la perte d'un de ces processus ne perd pas de données, alors que l'arrivée d'un nouveau processus peut soulager les responsabilités des autres (dans notre exercice, le stockage de données). On trouve donc les caractéristiques d'une application qui pourrait être déployée sur un cloud, en partant du principe qu'elle serait connectée à une base de données pour partager les tâches.

Notez qu'habituellement, ce genre de problématique est résolue par un service externe (ce qu'Hazelcast peut aussi faire) auquel les instances de l'application auraient eu à se connecter. On peut citer comme exemples de services alternatifs [Redis](#) ou [Memcached](#), ou les services offerts par les clouds publics ([AWS Elasticache](#), [Azure Redis Cache](#)...). On parle de systèmes de caches distribués.

Modifié le: lundi 23 mars 2026, 10:29

[Retour au cours](#)